6 MAGGIO 2026

Documentazione dell'implementazione Java + JML del sistema ascensore

MAFFEIS RICCARDO 1085706

Tema scelto	Ascensore a più piani con un'unica cabina
Obiettivo	Realizzazione e specifica formale del nucleo logico del modello ASM
Metodo	Implementazione Java annotata con contratti JML e verifica statica tramite ESC/OpenJML
Proprietà verificate	Invarianti di sicurezza, precondizioni, postcondizioni e invarianti di ciclo
Esito	Il nucleo logico risulta coerente con il modello ASM e parzialmente verificato tramite JML

Sommario

1. Introduzione ... 3
2. Obiettivo dell'implementazione Java ... 3
3. Differenze rispetto al modello ASM ... 4
4. Struttura del codice ... 5
5. Enumerazioni utilizzate ... 5
5.1 Direzione ... 5
5.2 StatoCabina ... 5
5.3 StatoPorte ... 6
5.4 StatoErrore ... 6
6. Classe Ascensore ... 6
7. Rappresentazione delle richieste attive ... 7
8. Invarianti JML della classe ... 8
8.1 Vincoli sullo stato numerico ... 8
8.2 Vincoli di non nullità ... 8
8.3 Invarianti di sicurezza sul movimento ... 9
8.4 Invarianti sul guasto ... 9
8.5 Invarianti sul sovraccarico ... 9
8.6 Invariante sul timer ... 9
9. Costruttore ... 9
10. Metodi di osservazione ... 10
11. Gestione delle richieste ... 10
12. Aggiornamento del numero di persone ... 11
13. Gestione del sovraccarico ... 11
14. Gestione del guasto ... 12
15. Gestione del timer di guasto ... 13
16. Gestione delle porte ... 13
17. Servizio del piano corrente ... 14
18. Stato di attesa ... 14
19. Ricerca delle richieste ... 14
19.1 esisteRichiesta ... 15
19.2 esisteRichiestaSopra ... 15
19.3 esisteRichiestaSotto ... 15
20. Scelta della direzione ... 16

21. Movimento della cabina 16
22. Verifica tramite ESC 17
23. Limiti della verifica 18
24. Conclusione 18

1. Introduzione

Questa documentazione descrive l'implementazione Java del nucleo logico del sistema ascensore modellato precedentemente mediante ASM.

L'obiettivo dell'implementazione Java non è quello di sostituire il modello ASM, ma di realizzare una versione eseguibile e verificabile del comportamento principale dell'ascensore. La classe Ascensore rappresenta quindi il nucleo operativo del sistema e viene annotata mediante JML per specificare formalmente proprietà, vincoli e contratti dei metodi principali.

La specifica JML è stata utilizzata per definire invarianti di classe, precondizioni, postcondizioni e proprietà sui cicli. La verifica statica tramite ESC consente di controllare automaticamente la coerenza di una parte significativa del codice rispetto alle proprietà dichiarate.

2. Obiettivo dell'implementazione Java

L'implementazione Java ha lo scopo di rappresentare i principali aspetti funzionali del sistema ascensore:

- gestione del piano corrente;
- gestione dello stato della cabina;
- gestione dello stato delle porte;
- gestione della direzione di movimento;
- gestione delle richieste attive;
- gestione del numero di persone presenti in cabina;
- gestione dello stato di sovraccarico;
- gestione dello stato di guasto;
- gestione del timer di ripristino;
- movimento della cabina di un piano alla volta.

La classe principale è:

`Ascensore`

Essa rappresenta il nucleo logico del sistema e contiene sia lo stato interno dell'ascensore sia i metodi che modificano tale stato.

3. Differenze rispetto al modello ASM

Il modello ASM descrive il comportamento dell'ascensore come una macchina astratta a stati. In ASM il sistema è definito tramite domini, funzioni monitored, funzioni controlled e regole di transizione.

L'implementazione Java, invece, organizza la logica del sistema in metodi separati. Ogni metodo rappresenta una parte del comportamento del modello ASM.

Nel modello ASM gli input esterni erano rappresentati da funzioni monitorate:

```
monitored chiamataPianoSu: Piano -> Boolean
monitored chiamataPianoGiu: Piano -> Boolean
monitored pulsanteInterno: Piano -> Boolean
monitored eventoGuasto: Boolean
monitored personeEntrate: Integer
monitored personeUscite: Integer
```

Nel codice Java questi input sono rappresentati da chiamate di metodo:

```
aggiungiRichiestaInterna(piano);
aggiungiChiamataSalita(piano);
aggiungiChiamataDiscesa(piano);
attivaGuasto();
aggiornaPersone(personeEntrate, personeUscite);
```

Inoltre, mentre il modello ASM contiene una regola principale che coordina l'intero comportamento del sistema, nella versione Java i singoli comportamenti sono separati in metodi indipendenti. Questa scelta rende più semplice la specifica JML e la verifica tramite ESC.

L'implementazione Java deve quindi essere considerata come una realizzazione parziale ma coerente del nucleo logico del modello ASM.

4. Struttura del codice

Il codice è organizzato nel package:

```
package progetto;
```

La classe principale è *Ascensore.java*.

Le enumerazioni utilizzate sono:

```
Direzione
StatoCabina
StatoPorte
StatoErrore
```

5. Enumerazioni utilizzate

5.1 Direzione

L'enumerazione *Direzione* rappresenta la direzione corrente dell'ascensore.

```
public enum Direzione {
	SU,
	GIU,
	NESSUNA
}
```

I valori indicano rispettivamente:

- SU: l'ascensore si sta muovendo o deve muoversi verso l'alto;
- GIU: l'ascensore si sta muovendo o deve muoversi verso il basso;
- NESSUNA: l'ascensore non ha una direzione attiva.

5.2 StatoCabina

L'enumerazione *StatoCabina* rappresenta lo stato operativo della cabina.

```
public enum StatoCabina {
	FERMA,
	IN_MOVIMENTO,
	BLOCCATA
}
```

I valori indicano:

- FERMA: la cabina è ferma;
- IN_MOVIMENTO: la cabina si sta muovendo;
- BLOCCATA: la cabina è bloccata a causa di un errore.

5.3 StatoPorte

L'enumerazione *StatoPorte* rappresenta lo stato delle porte.

```
public enum StatoPorte {
      APERTE,
      CHIUSE
}
```

I valori indicano:

- APERTE: le porte sono aperte;
- CHIUSE: le porte sono chiuse.

5.4 StatoErrore

L'enumerazione *StatoErrore* rappresenta la condizione di errore del sistema.

```
public enum StatoErrore {
      NESSUNO,
      OVERLOAD,
      GUASTO
}
```

I valori indicano:

- NESSUNO: il sistema non presenta errori;
- OVERLOAD: la cabina è in sovraccarico;
- GUASTO: il sistema è in stato di guasto tecnico.

6. Classe Ascensore

La classe *Ascensore* contiene lo stato principale del sistema e i metodi che implementano la logica dell'ascensore.

Le costanti principali sono:

```
public static final int PIANO_MINIMO = -1;
public static final int PIANO_MASSIMO = 4;
public static final int NUMERO_PIANI = 6;
public static final int CAPACITA_MASSIMA = 8;
public static final int TIMER_MASSIMO = 10;
```

Queste costanti definiscono:

- il piano minimo raggiungibile;
- il piano massimo raggiungibile;
- il numero totale di piani;
- la capacità massima della cabina;
- il valore massimo del timer di ripristino dopo un guasto.

Essendo dichiarate come public static final, tali costanti sono fissate direttamente dal linguaggio Java e non necessitano di ulteriori invarianti JML.

Lo stato interno dell'ascensore è rappresentato dai seguenti attributi:

```
private /*@ spec_public @*/ int pianoCorrente;
private /*@ spec_public @*/ int timer;
private /*@ spec_public @*/ int numeroPersone;

private /*@ spec_public @*/ StatoCabina statoCabina;
private /*@ spec_public @*/ StatoPorte statoPorte;
private /*@ spec_public @*/ Direzione direzione;
private /*@ spec_public @*/ StatoErrore statoErrore;

private /*@ spec_public @*/ final boolean[] richiesteAttive;
```

Gli attributi sono dichiarati private per proteggere lo stato interno della classe e impedire modifiche dirette non controllate dall'esterno.

L'annotazione JML:

```
/*@ spec_public @*/
```

consente comunque di usare questi campi nelle specifiche JML pubbliche.

7. Rappresentazione delle richieste attive

Nel modello ASM le richieste erano rappresentate come funzione:

```
richiesteAttive: Piano -> Boolean
```

Nel codice Java sono rappresentate tramite un array:

```
private final boolean[] richiesteAttive;
```

Poiché in Java gli array partono da indice 0, mentre i piani dell'ascensore vanno da -1 a 4, viene utilizzato il metodo:

```
indiceDelPiano(int piano)
```

Il metodo converte il numero del piano nell'indice dell'array.

Il contratto JML del metodo è:

```
//@ requires pianoValido(piano);
//@ ensures 0<=\result && \result<NUMERO_PIANI;
//@ ensures \result == piano - PIANO_MINIMO;
```

Questo garantisce che ogni piano valido venga convertito in un indice valido dell'array.

8. Invarianti JML della classe

La classe Ascensore definisce diversi invarianti JML. Gli invarianti rappresentano proprietà che devono essere sempre vere per ogni oggetto valido della classe.

8.1 Vincoli sullo stato numerico

```
//@ public invariant PIANO_MINIMO<=pianoCorrente && pianoCorrente<=PIANO_MASSIMO;
//@ public invariant 0<=timer && timer<=TIMER_MASSIMO;
//@ public invariant numeroPersone>=0;
```

Questi invarianti garantiscono che:

- il piano corrente sia sempre compreso tra il piano minimo e il piano massimo;
- il timer sia sempre compreso tra 0 e TIMER_MASSIMO;
- il numero di persone non sia mai negativo.

8.2 Vincoli di non nullità

```
//@ public invariant statoCabina != null;
//@ public invariant statoPorte != null;
//@ public invariant direzione != null;
//@ public invariant statoErrore != null;
//@ public invariant richiesteAttive != null;
//@ public invariant richiesteAttive.length == NUMERO_PIANI;
```

Questi invarianti garantiscono che gli stati dell'ascensore siano sempre definiti e che l'array delle richieste sia correttamente inizializzato.

8.3 Invarianti di sicurezza sul movimento

```
//@ public invariant statoCabina == StatoCabina.IN_MOVIMENTO ==> statoPorte == StatoPorte.CHIUSE;
//@ public invariant statoCabina == StatoCabina.IN_MOVIMENTO ==> statoErrore == StatoErrore.NESSUNO;
```

Questi invarianti formalizzano due proprietà fondamentali:

- la cabina può muoversi solo con le porte chiuse;
- la cabina può muoversi solo se non sono presenti errori.

8.4 Invarianti sul guasto

```
//@ public invariant statoErrore == StatoErrore.GUASTO ==> statoCabina == StatoCabina.BLOCCATA;
//@ public invariant statoErrore == StatoErrore.GUASTO ==> statoPorte == StatoPorte.CHIUSE;
//@ public invariant statoErrore == StatoErrore.GUASTO ==> direzione == Direzione.NESSUNA;
```

Quando il sistema è in stato GUASTO, la cabina deve essere bloccata, le porte devono essere chiuse e la direzione deve essere annullata.

8.5 Invarianti sul sovraccarico

```
//@ public invariant statoErrore == StatoErrore.OVERLOAD ==> statoCabina == StatoCabina.BLOCCATA;
//@ public invariant statoErrore == StatoErrore.OVERLOAD ==> statoPorte == StatoPorte.APERTE;
//@ public invariant statoErrore == StatoErrore.OVERLOAD ==> direzione == Direzione.NESSUNA;
```

Quando il sistema è in stato OVERLOAD, la cabina deve essere bloccata, le porte devono essere aperte e la direzione deve essere annullata.

8.6 Invariante sul timer

```
//@ public invariant timer>0 ==> statoErrore == StatoErrore.GUASTO;
```

Questo invariante stabilisce che il timer può essere positivo solo durante uno stato di guasto.

9. Costruttore

Il costruttore inizializza l'ascensore nello stato iniziale:

```
public Ascensore()
```

Lo stato iniziale è:

```
pianoCorrente = 0
timer = 0
numeroPersone = 0
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
statoErrore = NESSUNO
richiesteAttive = array di 6 elementi
```

Le postcondizioni JML garantiscono che l'oggetto venga creato in uno stato coerente:

```
//@ ensures pianoCorrente == 0;
//@ ensures timer == 0;
//@ ensures numeroPersone == 0;
//@ ensures statoCabina == StatoCabina.FERMA;
//@ ensures statoPorte == StatoPorte.CHIUSE;
//@ ensures direzione == Direzione.NESSUNA;
//@ ensures statoErrore == StatoErrore.NESSUNO;
//@ ensures richiesteAttive != null;
//@ ensures richiesteAttive.length == NUMERO_PIANI;
```

10. Metodi di osservazione

La classe contiene diversi metodi pure, utilizzati per leggere lo stato senza modificarlo:

```
getPianoCorrente()
getTimer()
getNumeroPersone()
getStatoCabina()
getStatoPorte()
getDirezione()
getStatoErrore()
```

11. Gestione delle richieste

La classe permette di aggiungere tre tipi di richieste:

```
aggiungiRichiestaInterna(int piano)
aggiungiChiamataSalita(int piano)
aggiungiChiamataDiscesa(int piano)
```

Questi metodi rappresentano rispettivamente:

- la pressione di un pulsante interno alla cabina;
- una chiamata esterna verso l'alto;
- una chiamata esterna verso il basso.

Tutti questi metodi richiedono che il piano sia valido e che il sistema non sia in stato di guasto:

```
//@ requires pianoValido(piano);
//@ requires statoErrore != StatoErrore.GUASTO;
//@ ensures richiesteAttive[indiceDelPiano(piano)] == true;
public void aggiungiRichiestaInterna(int piano)
```

In stato GUASTO, quindi, non vengono acquisite nuove richieste.

Questa scelta è coerente con il modello ASM, nel quale durante il guasto il sistema sospende l'acquisizione di nuove richieste.

12. Aggiornamento del numero di persone

Il metodo:

```
aggiornaPersone(int personeEntrate, int personeUscite)
```

aggiorna il numero di persone presenti in cabina.

Le precondizioni richiedono che:

```
//@ requires personeEntrate >= 0;
//@ requires personeUscite >= 0;
//@ requires statoPorte == StatoPorte.APERTE;
//@ requires statoCabina == StatoCabina.FERMA || statoCabina == StatoCabina.BLOCCATA;
```

Quindi il numero di persone può essere modificato solo se le porte sono aperte e la cabina è ferma o bloccata.

La postcondizione garantisce che il numero di persone non diventi mai negativo:

```
//@ ensures numeroPersone >= 0;
```

Il metodo è annotato con:

```
@CodeBigintMath
```

Questa annotazione viene usata per evitare falsi allarmi legati all'overflow degli interi durante la verifica statica con ESC.

13. Gestione del sovraccarico

Il metodo:

```
gestisciSovraccarico()
```

gestisce il passaggio allo stato OVERLOAD.

Se il numero di persone supera la capacità massima, il sistema entra in sovraccarico:

```
statoErrore = OVERLOAD
statoCabina = BLOCCATA
statoPorte = APERTE
direzione = NESSUNA
timer = 0
```

Se invece il sistema era in OVERLOAD e il numero di persone torna entro la capacità massima, il sistema ritorna operativo:

```
statoErrore = NESSUNO
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 0
```

Questo comportamento è coerente con il modello ASM, nel quale il sovraccarico blocca la cabina e mantiene le porte aperte.

14. Gestione del guasto

Il metodo:

```
attivaGuasto()
```

attiva lo stato di guasto tecnico.

Le precondizioni richiedono che il sistema non sia già in errore:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
```

Quando viene attivato il guasto, il sistema passa nello stato:

```
statoErrore = GUASTO
statoCabina = BLOCCATA
statoPorte = CHIUSE
direzione = NESSUNA
timer = TIMER_MASSIMO
```

Questo è coerente con il modello ASM, nel quale il guasto blocca la cabina, chiude le porte e inizializza un timer di ripristino.

15. Gestione del timer di guasto

Il metodo:

```
gestisciTimerGuasto()
```

viene utilizzato quando il sistema si trova nello stato GUASTO.

Se il timer è maggiore di zero, viene decrementato:

```
timer = timer - 1;
```

Se invece il timer è uguale a zero, il sistema viene ripristinato:

```
statoErrore = NESSUNO
statoCabina = FERMA
statoPorte = CHIUSE
direzione = NESSUNA
timer = 0
```

Il contratto JML descrive formalmente entrambi i casi.

16. Gestione delle porte

Il metodo:

```
chiudiPorte()
```

chiude le porte dell'ascensore.

Può essere eseguito solo se:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
//@ requires statoCabina == StatoCabina.FERMA;
//@ requires statoPorte == StatoPorte.APERTE;
```

Quindi le porte vengono chiuse solo se il sistema è operativo, la cabina è ferma e le porte sono aperte.

17. Servizio del piano corrente

Il metodo:

```
serviPianoCorrente()
```

serve la richiesta associata al piano corrente.

Le postcondizioni garantiscono che:

```
statoCabina = FERMA
statoPorte = APERTE
direzione = NESSUNA
richiesta del piano corrente = false
```

Il metodo rappresenta il momento in cui l'ascensore arriva al piano richiesto, apre le porte e rimuove la richiesta attiva.

18. Stato di attesa

Il metodo:

```
mettiInAttesa()
```

porta l'ascensore nello stato di attesa.

Il metodo richiede che non siano presenti errori:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
```

Le postcondizioni garantiscono:

```
statoCabina = FERMA
direzione = NESSUNA
```

Questo metodo rappresenta il comportamento dell'ascensore quando non ci sono richieste da servire.

19. Ricerca delle richieste

La classe contiene tre metodi per analizzare le richieste attive:

```
esisteRichiesta()
esisteRichiestaSopra()
esisteRichiestaSotto()
```

Questi metodi sono dichiarati pure, perché leggono lo stato ma non lo modificano.

19.1 esisteRichiesta

Il metodo:

`esisteRichiesta()`

verifica se esiste almeno una richiesta attiva in qualsiasi piano.

La postcondizione JML è:

```
//@ ensures \result <==> (\exists int i; 0 <= i && i < NUMERO_PIANI; richiesteAttive[i]);
```

Il ciclo è annotato con invarianti di ciclo:

```
//@ loop_invariant 0 <= i && i <= NUMERO_PIANI;
//@ loop_invariant (\forall int j; 0 <= j && j < i; richiesteAttive[j] == false);
```

19.2 esisteRichiestaSopra

Il metodo:

`esisteRichiestaSopra()`

verifica se esiste una richiesta attiva a un piano superiore rispetto al piano corrente.

La postcondizione JML è:

```
//@ ensures \result <==>
//@     (\exists int p; pianoCorrente < p && p <= PIANO_MASSIMO;
//@         richiesteAttive[indiceDelPiano(p)]);
```

19.3 esisteRichiestaSotto

Il metodo:

`esisteRichiestaSotto()`

verifica se esiste una richiesta attiva a un piano inferiore rispetto al piano corrente.

La postcondizione JML è:

```
//@ ensures \result <==>
//@     (\exists int p; PIANO_MINIMO <= p && p < pianoCorrente;
//@         richiesteAttive[indiceDelPiano(p)]);
```

20. Scelta della direzione

Il metodo:

```
scegliDirezione()
```

decide la direzione dell'ascensore in base alle richieste attive.

Il metodo può essere eseguito solo se il sistema non presenta errori:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
```

La logica implementata è la seguente:

- se non esistono richieste, la direzione diventa NESSUNA;
- se la direzione corrente è SU e ci sono richieste sopra, la direzione resta SU;
- se la direzione corrente è SU ma non ci sono richieste sopra, il sistema cerca richieste sotto;
- se la direzione corrente è GIU e ci sono richieste sotto, la direzione resta GIU;
- se la direzione corrente è GIU ma non ci sono richieste sotto, il sistema cerca richieste sopra;
- se la direzione corrente è NESSUNA, il sistema sceglie una direzione in base alle richieste disponibili.

Questo metodo è il più articolato dal punto di vista logico, perché contiene più condizioni ramificate e dipende dai metodi ausiliari di ricerca delle richieste.

Durante la verifica ESC, questo metodo può risultare più difficile da dimostrare completamente, soprattutto a causa dell'uso di metodi pure all'interno di postcondizioni con \old.

Questo non indica necessariamente un errore nel comportamento, ma rappresenta un limite pratico della verifica automatica.

21. Movimento della cabina

Il metodo:

```
muoviDiUnPiano()
```

sposta l'ascensore di un piano nella direzione corrente.

Può essere eseguito solo se:

```
//@ requires statoErrore == StatoErrore.NESSUNO;
//@ requires statoPorte == StatoPorte.CHIUSE;
```

Quindi l'ascensore può muoversi solo senza errori e con le porte chiuse.

Se la direzione è SU e il piano corrente è minore del piano massimo, il piano viene incrementato:

```
pianoCorrente = pianoCorrente + 1;
statoCabina = StatoCabina.IN_MOVIMENTO;
```

Se la direzione è GIU e il piano corrente è maggiore del piano minimo, il piano viene decrementato:

```
pianoCorrente = pianoCorrente - 1;
statoCabina = StatoCabina.IN_MOVIMENTO;
```

Se non è possibile muoversi, la cabina viene fermata e la direzione viene annullata.

22. Verifica tramite ESC

La verifica statica è stata eseguita mediante ESC di OpenJML.

L'obiettivo della verifica è controllare che i metodi della classe rispettino le specifiche JML definite tramite:

- invarianti di classe;
- precondizioni;
- postcondizioni;
- annotazioni sui cicli;
- metodi pure.

La verifica ha permesso di individuare e correggere alcune incoerenze, in particolare relative alla gestione degli stati di errore.

23. Limiti della verifica

La verifica tramite ESC è stata applicata al nucleo logico del sistema, ma non tutte le parti risultano necessariamente dimostrabili automaticamente.

In particolare, il metodo:

```
scegliDirezione()
```

contiene una logica decisionale più complessa e utilizza metodi ausiliari come:

```
esisteRichiesta()
esisteRichiestaSopra()
esisteRichiestaSotto()
```

Questa struttura può rendere difficile per il verificatore dimostrare tutte le postcondizioni, soprattutto quando sono presenti espressioni con \old.

Per questo motivo, tale metodo può essere considerato corretto dal punto di vista implementativo e coerente con il modello ASM, ma la sua validazione completa può essere demandata ai test JUnit e ai successivi test di sistema.

24. Conclusione

L'implementazione Java della classe Ascensore rappresenta il nucleo logico principale del sistema ascensore modellato in ASM.

La specifica JML permette di formalizzare le principali proprietà di sicurezza e coerenza dello stato, tra cui:

- validità del piano corrente;
- validità del timer;
- non negatività del numero di persone;
- assenza di valori nulli negli stati;
- movimento consentito solo con porte chiuse;
- movimento consentito solo in assenza di errore;
- comportamento corretto in stato di guasto;
- comportamento corretto in stato di sovraccarico;
- gestione corretta delle richieste attive.

Il codice Java non sostituisce il modello ASM completo, ma ne implementa una parte significativa e verificabile. La combinazione tra ASM, AVALLA, ATGT, AsmetaSMV, Java,

JML e successivi test JUnit consente di coprire sia la modellazione formale astratta, sia l'implementazione concreta del sistema.

La parte Java/JML può quindi essere considerata coerente con il modello formale e adeguata come nucleo implementativo del progetto.